



NAMES: TELIMBERE Turamyeyezu Aime Christian Divin 28828
ISHIMWE Gad 29062
MUYANGO Hirwa Herve 29276
MUTANGANA Joseph 29061
ISHIMWE Darius 29368

GROUP: C
ASSIGNMENT GROUP 8

LOAN SWITCH MANAGEMENT SYSTEM

Project Foundation, Organization Context & Software Development Plan

Prepared for: Commercial Bank | Financial Services Sector
Software Quality Assurance & Project Management Coursework

Table of Contents

1. Project Foundation & Organization Context	3
2. Software Development Plan (SDP)	5
3. Synthesis & Comparative Insights	9
4. Software Quality Assurance Plan	10
5. Comparison Between Software Development Plan (SDP) and Software Quality Assurance Plan (SQAP)	15
6. 12-Month Project Schedule (Gantt Chart).....	17
7. How the Software Development Plan and the Software Quality Assurance Plan Work Together Throughout the Software Development Life Cycle (SDLC).....	19
8. Effects If the Project Changes from a 12-Month External Client Project to a 2-Month Internal Organizational Project	23
9. Consequences of Proceeding Without a Software Development Plan	24
10. Consequences of Proceeding Without a Software Quality Assurance Plan.....	26
11. Conclusion.....	29

Question 1: Project Foundation & Organization Context

1. Description of the Selected Organization

- **Organization:** Commercial Bank (referred to as “Bank”).
- **Sector:** Financial Services / Retail & Corporate Banking.
- **Operational Scale:** High-volume customer base across branches and digital channels, requiring real-time transaction reconciliation, automated ledger updates, and strict compliance governance.

2. Organization’s Business Activities

- Providing retail, corporate, and institutional lending products (variable/fixed-rate loans, term loans, mortgages).
- Financial risk and treasury management, including interest rate exposure balancing, liquidity monitoring, and yield curve optimization.
- Regulatory compliance reporting under central bank policies, Basel III capital adequacy standards, and IFRS 9 financial instrument accounting.

3. Business Problem or Opportunity

- **Business Problem:** Manual, paperwork-heavy processes for transferring a client's existing loan from a separate, external commercial bank (the client's current lender) to the Bank result in slow approval turnaround times, operational bottlenecks, high paperwork expenses, and human error in payoff-balance and settlement calculations. Legacy API limitations and uncoordinated manual data exchange between the two independent banking institutions also risk data inconsistencies between the transferred loan facility and the Bank's core banking ledgers.
- **Opportunity:** Automate the loan-switching process via a self-service or officer-assisted digital channel (Loan Switch Management System) that transfers a client's loan away from their current, separate commercial bank and onto the Bank's books. This reduces operational costs, enhances customer acquisition and retention by making it easy for clients to move their loan to the Bank, and automates credit/compliance checks.

4. Proposed Software Solution

- **System Name:** Loan Switch Management System (LSMS) / LoanSwitch Central.
- **Solution Summary:** An enterprise web and web-API application that automates eligibility verification for the client's switch request, calculates the outstanding payoff balance owed to the client's Originating Bank (a separate, independent commercial bank), coordinates settlement and collateral/lien transfer between the two distinct banking institutions, routes loan-switch approvals to the Bank's credit/compliance officers, and books the loan onto the Bank's core banking ledgers seamlessly via secure APIs.

5. Project Objectives

- **Timeline & Governance:** Successfully execute and deploy the LSMS within a strict 12-month SDLC under full approval from the Steering Committee.
- **Functional Automation:** Automate 100% of eligibility rule evaluations and payoff-balance/settlement calculations for incoming loan switches.
- **Core Integration:** Establish zero-variance integration with the Bank's Core Banking System, secure payoff/settlement interfaces with clients' Originating Banks (separate external commercial banks), and Credit Bureau APIs.

- **Compliance & Security:** Achieve 100% compliance with Basel III, IFRS 9, and central bank lending directives, and pass 100% of OWASP Top 10 security checks prior to go-live.

6. Expected Benefits

- **Efficiency & Cost Reduction:** Eliminates physical branch-visit requirements and paper processing, drastically cutting processing cycles and operational overhead.
- **Data Integrity:** Eliminates financial reconciliation variance between front-end loan-switch/payoff requests and core accounting ledgers.
- **Risk Control:** Enforces real-time role-based authorization, automated audit logging, and continuous compliance verification.

Question 2: Software Development Plan (SDP)

1. Project Objectives

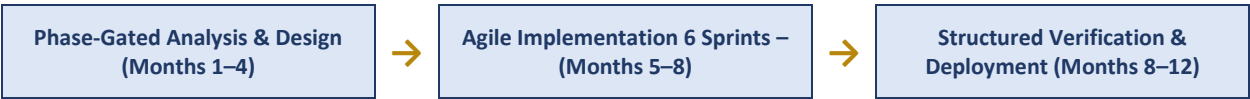
- Deliver a fully functional, secure, and high-performing Loan Switch Management System in 12 months.
- Ensure total alignment between project management commitments (SDP) and quality control gates (SQAP).

2. Project Scope

In-Scope	Out-of-Scope
Self-service customer loan-switch requests (transferring an existing loan facility from a separate, external commercial bank to the Bank, including payoff calculation, settlement, and collateral/lien transfer between the two institutions)	Direct underwriting of entirely new loan applications (restricted purely to switching/transferring existing facilities from another bank)
Automated eligibility evaluation engine and payoff/settlement calculation engine	Unapproved mid-project scope additions (e.g., mobile apps, external government database links) unless authorized via formal Change Control
Internal approval workflow dashboard for Credit and Compliance Officers	
API integrations with Core Banking Systems, Credit Bureaus, and Notification Services	
Historical data migration, role-based access control (RBAC), and automated audit trails	

3. Development Methodology — Hybrid Model

Waterfall phase-gating combined with Agile sprints:



- Phase-Gated Analysis & Design (Months 1–4): structured requirement analysis and architectural design to satisfy regulatory, contract, and compliance reviews.
- Agile Implementation (Months 5–8): six 2-week Agile Sprints for iterative coding, peer code reviews, and unit testing.
- Structured Verification & Deployment (Months 8–12): SIT, UAT, security/penetration testing, data cutover, and hypercare.

4. Project Organization

The multidisciplinary project team is structured with clear accountability across functional and technical domains:

Business Analyst	Technical Lead & Developers	Database Administrator	QA Lead / Testers	Compliance & Risk Liaison
------------------	-----------------------------	------------------------	-------------------	---------------------------

- **Project Manager:** Project schedule execution, budget tracking, resource management, and stakeholder communication.

- **Business Analyst:** Scope definition, requirements gathering, and traceability matrix maintenance.
- **Technical Lead & Software Developers:** Architecture implementation, API development, financial engine logic, peer code reviews.
- **Database Administrator (DBA):** Database schema design, trial data migration, ledger alignment.
- **QA Lead & Software Testers:** SQAP execution, SIT/UAT testing, defect logging and verification.
- **Compliance & Risk Liaison:** Basel III / IFRS 9 audit review, data privacy verification.

5. Work Breakdown Structure (WBS)

■ 1.0 Loan Switch Management System (LSMS)

- ▶ 1.1 Project Initiation & Governance
 - 1.1.1 SDP & SQAP Baseline
 - 1.1.2 Proposal / Contract Stage Review Clearance
- ▶ 1.2 Requirements Analysis & Specification
 - 1.2.1 Business Requirements Elicitation
 - 1.2.2 Requirements Traceability & Sign-Off
- ▶ 1.3 System Architecture & Design
 - 1.3.1 Database & API Schema Design
 - 1.3.2 UI/UX Wireframing & Security Architecture
- ▶ 1.4 System Implementation (Sprints 1–6)
 - 1.4.1 Eligibility & Calculation Engine
 - 1.4.2 Core Banking Integration APIs
 - 1.4.3 Approval Workflows & Reporting Dashboards
- ▶ 1.5 System Verification & Testing
 - 1.5.1 Integration & Security Penetration Testing
 - 1.5.2 User Acceptance Testing (UAT) Sign-Off
- ▶ 1.6 Production Deployment & Post-Go-Live
 - 1.6.1 Trial & Final Data Migration Cutover
 - 1.6.2 Hypercare Support & Project Closure

6. Project Schedule (Gantt Summary)

Aligned directly with the 12-month baseline timeline:

Phase / Activity	Start	End	Key Deliverables / Milestones
1. Project Initiation	M1	M1	SDP & SQAP baseline approval; Contract Stage 1/2 clearance
2. Requirements Analysis	M2	M3	Software Requirements Specification (SRS) sign-off
3. System Design	M3	M4	Architecture, DB Schema, and UI design sign-off
4. Implementation (Sprints 1–6)	M5	M8	Loan-switch payoff/settlement engine, inter-bank API integrations built
5. Testing (SIT / Security / UAT)	M8	M10.5	UAT sign-off; zero critical defects achieved

6. Deployment & Data Migration	M10	M11	Historical data cutover; Production go-live
7. Stabilization & Maintenance	M11	M12	Hypercare completion and formal project closure

Visual Timeline (Gantt Bars by Month)

Phase	M1	M2	M3	M4	M5	M6	M7	M8	M9	M10	M11	M12
Initiation												
Requirements												
System Design												
Implementation												
Testing (SIT/UAT)												
Deployment & Migration												
Stabilization												

7. Resource Allocation

Human Resources	Infrastructure & Tools
1 Project Manager	Automated CI/CD build pipelines
2 Business Analysts	Static code analysis tools (SonarQube)
1 Technical Lead + 4 Software Developers	Issue tracking system (Jira)
1 Database Administrator (DBA)	Dedicated Staging / UAT servers
1 UI/UX Designer	
1 QA Lead + 2 Testers	
1 Compliance / Security Specialist	

8. Budget Assumptions

- **Total Estimated Budget:** \$950,000 baseline.

Budget Category	% of Total	Amount (USD)
Personnel & Software Engineering	60%	\$570,000
Infrastructure, Hardware & API Licenses	15%	\$142,500
Quality Assurance, Security Audits & Testing	15%	\$142,500
Contingency Buffer (Risk/Change Mgmt.)	10%	\$95,000
TOTAL ESTIMATED BUDGET	100%	\$950,000

9. Risk Management Plan

Incorporating the prioritized risk register and matrix:

ID	Risk Description	Likelihood / Impact	Mitigation Strategy
R1	Regulatory non-compliance (Basel III / IFRS 9)	Med / High	Engage Compliance Liaison from Month 1; conduct phase-gate audits
R2	Core Banking integration failure via legacy APIs	Med / High	Execute early API Proof-of-Concept in Months 2–3
R3	Data migration errors during historical record cutover	Med / Med	Run automated trial migrations with verification scripts in Month 10
R4	Key technical staff attrition mid-project	Low-Med / High	Strict documentation discipline; cross-train developers
R5	Cybersecurity breach exposing customer data	Low / High	Enforce data encryption; independent penetration testing
R6	Uncontrolled scope creep from business stakeholders	Med-High / Low-Med	Strict formal Change Control Process via Steering Committee

10. Communication Plan

Stakeholder Group	Communication Vehicle	Frequency	Key Objective / Output
Steering Committee	Executive Progress Meeting	Monthly	Review milestone status, budget, and scope changes
Development & QA Team	Daily Scrum / Sprint Review	Daily / Biweekly	Task progress, defect resolution, sprint velocity
Business / Compliance	Phase-Gate Review Meetings	Milestone Ends	SRS sign-off, UAT approvals, compliance checks
External / Internal Audit	SQA Audit Summary Report	Quarterly	Process compliance and contract review verification

Synthesis & Comparative Insights

To ensure robust planning, key lessons from related software project analyses have been integrated into this framework:

1. SDP vs. SQAP Integration

While the SDP establishes what will be built, who will build it, and when activities will occur, it is bound directly to the SQAP (Software Quality Assurance Plan), which defines how quality is verified. Every milestone phase-gate in the SDP schedule requires a formal SQA review sign-off (e.g., SRS review, code walkthrough, security inspection) before moving to the next stage.

2. Contract Review Safeguards (Preventing CFV-Style Project Failures)

To avoid severe schedule and budget slippages — such as those seen in project case studies like CFV Corporation, where unreviewed assumptions led to an 18-month delay and cost overruns — this SDP mandates a two-stage Contract Review discipline:

- **Stage 1 (Proposal Draft Review):** Conducted prior to proposal submission to verify that requirements are complete and measurable, estimates are based on historical data, and technical feasibility is proven.
- **Stage 2 (Contract Draft Review):** Conducted after customer negotiations to verify that no unapproved deviations or unbudgeted scope additions were introduced into the final agreement.

3. Adaptability to Compressed Timelines (2-Month Internal Scope)

If the system scale were compressed from a 12-month external client project to a 2-month internal project, the SDP would streamline lightweight project documentation, shorten schedule timelines, and reduce formal committee reporting. However, core SQA activities — such as requirements review, security checks, unit/integration testing, and defect management — remain non-negotiable to protect data integrity and regulatory compliance.

4. Software Quality Assurance Plan

1.4.1 Quality Objectives

The overarching quality objective of the LoanSwap Central project is to deliver a banking software system that is accurate, secure, reliable, and compliant with financial regulations, while meeting the functional and non-functional needs of Bank and its customers. Specific, measurable quality objectives are set out below.

Objective	Description	Target
Functional correctness	The system must correctly execute all loan swap calculations, eligibility rules, and approval workflows as specified in the requirements.	100% of critical business rules verified; zero critical defects at go-live
Data accuracy & integrity	Interest recalculations, repayment schedules, and ledger updates must reconcile exactly with core banking figures.	0% variance in financial reconciliation tests
Security & confidentiality	Customer financial data and transactions must be protected against unauthorised access, in line with data protection and banking regulations.	100% of OWASP Top 10 checks passed; zero critical vulnerabilities at release
Reliability & availability	The system must remain available during banking hours and recover gracefully from failures.	99.5% uptime during business hours; recovery time objective (RTO) under 4 hours
Usability	Bank staff and customers should be able to complete a loan swap request with minimal training and errors.	Task success rate of at least 90% in usability testing
Performance & scalability	The system must process loan swap requests within acceptable response times under expected transaction volumes.	95% of transactions processed within 3 seconds under peak load
Regulatory compliance	The system must satisfy central bank and internal audit requirements for lending systems.	100% compliance checklist sign-off before deployment
Customer satisfaction	Customers should experience a smooth, transparent loan swap process.	Post-implementation satisfaction score of at least 4 out of 5

2.4.2 Applicable Standards and Procedures

LoanSwap system will be developed and quality-assured against internationally recognised software engineering standards and policies, adapted to the context of a financial services application.

1. 4.2.1 International and Industry Standards

- ISO/IEC 25010:2011 (Systems and Software Quality Requirements and Evaluation, SQuARE) — used to define and evaluate the product quality characteristics of functionality, reliability, usability, security, maintainability, performance and scalability.
- ISO/IEC/IEEE 29119 — Software Testing standard, used to structure the test strategy, test plans, and test documentation.
- IEEE 730 — Standard for Software Quality Assurance Processes, used as the structural basis for this SQAP.
- ISO 9001 — Quality Management System principles, applied to document control, process consistency, and continual improvement..

2. 4.2.2 Regulatory and Organisational Procedures

- Central bank lending and consumer protection guidelines applicable to loan restructuring and interest rate disclosures.
- Data protection legislation (e.g. data privacy and banking secrecy regulations) governing the handling of customer financial data.
- Bank's internal IT change management, access control, and audit trail policies.
- The bank's Software Development Life Cycle (SDLC) governance procedures and sign-off gates.

3.4.3 Review and Inspection Activities

Reviews and inspections are used to detect defects early, before they propagate into code and testing, which is significantly cheaper than correcting them after deployment. The following review and inspection activities will be conducted throughout the SDLC.

Review Type	Work Product Reviewed	Timing
Requirements review	Business Requirements Document, Software Requirements Specification	End of Requirements Analysis phase
Design review	System architecture, database schema, interest-calculation logic, UI/UX wireframes	End of System Design phase
Code review / walkthrough	Source code for loan-swap calculation modules, approval workflow, integration APIs	Continuously during Implementation, before each merge
Formal technical inspection	High-risk modules: interest engine, core banking link, security	Before each module is promoted to test environment
Test case review	Test plans, test scripts, test data sets	Before Testing phase begins
Pre-deployment (go/no-go) review	UAT results, outstanding defects, compliance checklist	End of Testing phase, before Deployment

4.4.4 Testing Strategy

Testing for LoanSwap System follows a layered strategy aligned with ISO/IEC/IEEE 29119, moving from isolated component testing to full business-process validation with the client, with particular emphasis on financial calculation accuracy and security given the banking context.

Test Level	Objective	Responsibility	Example Focus Areas
Unit testing	Verify individual functions and modules work correctly in isolation	Software Developers	Interest rate formulas, eligibility rule functions, input validation

Integration testing	Verify that modules and external systems interact correctly.	Software Testers, Database Administrator	Core banking system integration, credit bureau API, notification service
System testing	Verify the complete system meets functional and non-functional requirements.	QA Engineer, Software Testers	End-to-end loan swap workflow, performance under load, security testing
Regression testing	Ensure new changes do not break existing functionality.	Software Testers (supported by automated scripts)	Re-run of core financial and workflow test suites after each release
User Acceptance Testing (UAT)	Confirm the system meets business needs from the client's perspective.	Bank staff (loan officers, compliance), Business Analyst	Branch staff processing real-world loan swap scenarios
Security testing	Identify vulnerabilities before release.	Security Specialist / QA Engineer	Authentication, authorisation, data encryption, penetration testing
Performance testing	Confirm the system meets response-time and load targets.	QA Engineer	Concurrent users at month-end processing peaks

5.4.5 Quality Metrics

Quality metrics will be collected throughout the project to provide objective, measurable evidence of product and process quality, and to support data-driven decisions at each project gate.

Metric	Definition	Target
Defect density	Number of defects per 1,000 lines of code (or per module) found during testing.	Fewer than 3 defects per 1,000 lines of code at system test exit
Test case pass rate	Percentage of executed test cases that pass.	At least 95% before go-live
Requirements coverage	Percentage of requirements mapped to at least one test case.	100%
Critical defect count at release	Number of unresolved critical/high-severity defects at go-live.	Zero
Mean time to resolve (MTTR)	Average time to resolve a reported defect once logged.	Critical: 24 hours; High: 3 days; Medium: 7 days
System availability	Percentage uptime during business hours in production.	99.5%
Customer-reported defects	Number of defects reported by customers/staff within 90 days of go-live.	Downward trend, benchmarked against prior releases

6.4.6 Defect Management Process

Defect Severity Classification

Severity	Description	Example	Response Target
Critical	System unusable, data corruption, or incorrect financial calculation.	Loan swap produces an incorrect repayment amount	Fix within 24 hours; blocks release

High	Major function impaired with no acceptable workaround.	Approval workflow fails to route to compliance officer	Fix within 3 business days
Medium	Function impaired but a workaround exists.	Report export requires manual refresh	Fix within 1–2 weeks / next release
Low	Minor/cosmetic issue with no functional impact.	Misaligned label on a form	Fix in a future maintenance release

7.4.7 Quality Audits

Audit	Scope	Frequency	Conducted By
Process compliance audit	Verifies that SDLC activities (reviews, testing, documentation) are being followed as planned.	Monthly	Independent QA Lead / internal audit representative
Configuration and change management audit	Verifies version control, change requests, and release records are correctly maintained.	Quarterly / per release	Project Manager, QA Lead
Security and compliance audit	Verifies the system meets data protection, access control, and regulatory requirements.	Before each major release and before go-live	Security Specialist, Bank Compliance Officer
Final pre-release quality audit	Comprehensive review of test results, defect status and documentation before deployment.	End of Testing phase	Steering Committee, QA Lead, Client Sponsor

8.4.8 Acceptance Criteria

The following criteria must be satisfied before Bank accepts LoanSwap system for production deployment. Acceptance will be formally recorded through client sign-off at the pre-deployment review.

- All requirements in the Software Requirements Specification are implemented and traced to at least one passed test case.
- 100% of critical and high-severity defects are resolved and verified; no more than an agreed threshold of low-severity defects remain open.
- User Acceptance Testing has been completed by nominated bank staff with a pass rate of at least 95% and formal UAT sign-off obtained.
- Security testing confirms no critical or high vulnerabilities remain unresolved, and access controls enforce role-based permissions correctly.
- Performance testing confirms the system meets agreed response-time and concurrent-user targets under simulated peak load.
- Regulatory and compliance checklist items have been reviewed and signed off by the bank's Compliance Officer.

9.4.9 Continuous Improvement Plan

1. Feedback and Learning Loops

- End-of-phase retrospectives after each major SDLC phase to capture lessons learned and adjust processes for the next phase.
- Trend analysis of quality metrics (defect density, MTTR, test pass rates) each sprint/phase to identify recurring root causes.

2. 4.10 Process and Standards Updates

- Review checklists, test case libraries, and coding standards are updated whenever a recurring defect pattern is identified.
- The SQAP itself is a controlled document, reviewed and updated at each phase gate to reflect lessons learned and any changes in regulatory requirements.

3. 4.11 Post-Implementation Monitoring

- Production incidents and customer-reported defects are logged and fed back into the defect management process and quality metrics.
- A post-implementation review is conducted approximately 90 days after go-live to assess whether quality objectives (Section 3.1) were achieved and to plan improvements for future releases.

5. Comparison Between Software Development Plan (SDP) and Software Quality Assurance Plan (SQAP)

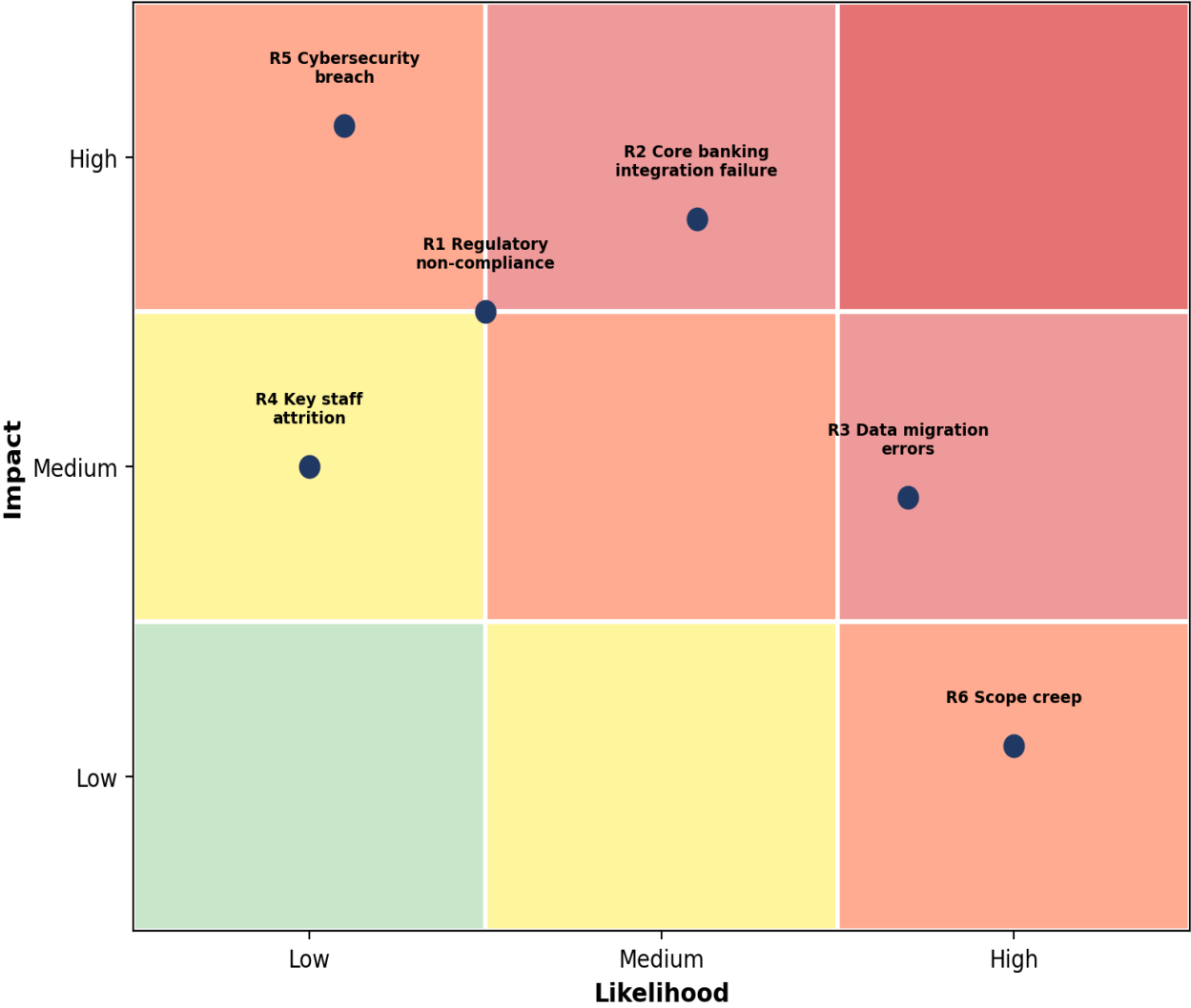
Comparison Point	Software Development Plan (SDP)	Software Quality Assurance Plan (SQAP)
Purpose	Defines how the software project will be planned and executed.	Defines how software quality will be achieved and maintained.
Main Focus	Project management and development activities.	Software quality and defect prevention.
Scope	Covers the entire software development process.	Covers quality assurance activities throughout the SDLC.
Objective	Deliver the project on time, within budget, and according to scope.	Ensure the software meets quality standards and customer expectations.
Contents	Scope, schedule, resources, budget, risks, communication, WBS.	Quality objectives, testing, reviews, audits, quality metrics, defect management.
Responsibility	Mainly managed by the Project Manager.	Mainly managed by the Quality Assurance (QA) Manager or QA Team.
Development Methodology	Specifies Agile, Waterfall, Scrum, etc.	Ensures quality practices are followed regardless of methodology.
Deliverables	Project plans, schedules, milestones, and software products.	Test reports, audit reports, defect reports, quality records.
Success Measure	Completion of the project within scope, time, and budget.	Delivery of reliable, secure, and defect-free software.
Timing	Prepared before development begins and updated as needed.	Prepared before development begins and used throughout the project life.

5.1 Project Risk Register

#	Risk Description	Possible Impact	Likelihood / Risk Level	Mitigation Strategy
1	Regulatory non-compliance: the system fails to meet Basel III / IFRS 9 reporting requirements	Regulatory penalties, delayed go-live, reputational damage	Medium likelihood / High impact	Engage Compliance & Risk Liaison from Month 1; conduct

				phase-gate compliance reviews
2	Core Banking System integration failure due to legacy API limitations	Delayed implementation, data inconsistency between systems	Medium likelihood / High impact	Conduct early integration proof-of-concept in Month 2–3; allocate dedicated integration budget
3	Data migration errors when moving historical loan/swap records	Inaccurate financial reporting, reconciliation failures post go-live	Medium likelihood / Medium impact	Perform trial data migrations with validation scripts before final cutover
4	Key technical staff attrition (e.g., Technical Lead or DBA resigning mid-project)	Loss of project knowledge, schedule delays, reduced code quality	Low-Medium likelihood / High impact	Maintain documentation discipline, cross-train team members, retain a bench resource
5	Cybersecurity breach exposing sensitive financial or customer data	Financial loss, regulatory sanctions, severe reputational damage	Low likelihood / High impact	Conduct security testing, encryption, and independent penetration testing before go-live
6	Scope creep from Treasury or Compliance requesting new features mid-project	Budget overrun, schedule delay	Medium-High likelihood / Medium impact	Enforce a formal Change Control Process requiring Steering Committee approval for scope changes

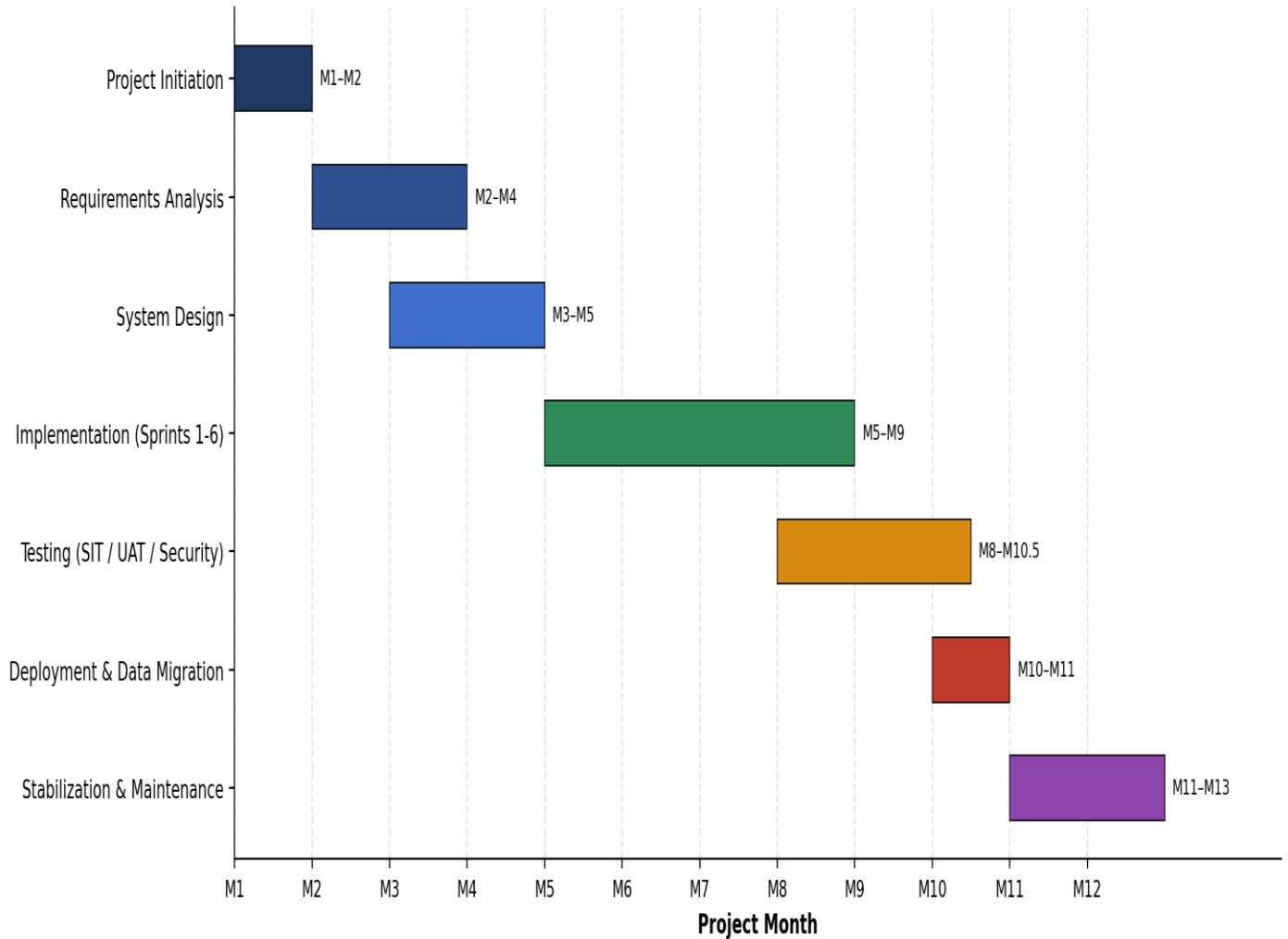
Figure 5: Project Risk Matrix – LSMS Project



6. 12-Month Project Schedule (Gantt Chart)

The Gantt chart below presents the major phases of the LSMS project across its 12-month duration, from Project Initiation through Maintenance.

Figure 3: 12-Month Project Gantt Chart – LSMS Project



Phase	Start	End	Key Milestone
Project Initiation	Month 1	Month 1	SDP & SQAP approved by Steering Committee
Requirements Analysis	Month 2	Month 3	Requirements Specification signed off
System Design	Month 3	Month 4	Architecture & database design approved
Implementation	Month 5	Month 8	Core swap engine and integrations complete
Testing	Month 8	Month 10 (mid)	UAT sign-off achieved
Deployment	Month 10	Month 10	Production go-live
Maintenance	Month 11	Month 12	Hypercare complete, project closure

7. How the Software Development Plan and the Software Quality Assurance Plan Work Together Throughout the Software Development Life Cycle (SDLC)

7.1 Division of Responsibility, Convergence of Purpose

The SDP is primarily the Project Manager's instrument. It carries the Work Breakdown Structure, the resource allocation across the Project Manager, Business Analysts, Technical Lead, developers, DBA, QA Lead and Compliance and Risk Liaison, the \$950,000 budget split across personnel, infrastructure, quality assurance and contingency, the risk register, and the communication plan that reports progress to the Steering Committee monthly. The SQAP is primarily the QA Lead's instrument. It carries the quality objectives (zero-variance financial reconciliation, 100% OWASP Top 10 pass rate, 100% regulatory compliance sign-off), the applicable standards (ISO/IEC 25010, ISO/IEC/IEEE 29119, IEEE 730, ISO 9001), the review and inspection schedule, the layered testing strategy from unit testing through UAT, the quality metrics, the defect severity classification and the formal acceptance criteria. These two instruments converge at a single mechanism: the phase gate. Each SDP phase produces a defined deliverable — a Software Requirements Specification, an architecture and database design, working code, a tested system, a deployed production system — and the SQAP defines the review or test that must pass before that deliverable is accepted and the SDP schedule is permitted to advance to the next phase.

7.2 Phase-by-Phase Interaction Across the SDLC

The table below maps each SDLC phase in LoanSwap Central's schedule to the corresponding SDP planning activity and SQAP verification activity, and the sign-off artefact that formally links them.

SDLC Phase	SDP Contribution	SQAP Contribution	Gate / Sign-off
Initiation	Establishes project charter, WBS, schedule, budget, team roles and the SDP/SQAP baseline itself	Confirms SQAP scope, standards (ISO/IEC 25010, IEEE 730, ISO/IEC/IEEE 29119) and review cadence are agreed	SDP & SQAP approved by Steering Committee
Requirements Analysis	Business Analyst elicits and documents requirements per the schedule in the WBS	Requirements review checks completeness, testability and traceability before design starts	SRS sign-off
System Design	Technical Lead and DBA produce architecture, API and database schema per the resourcing plan	Design review verifies the architecture satisfies functional and non-functional requirements, including security architecture	Architecture & DB design sign-off
Implementation (Sprints 1-6)	Developers build the eligibility, calculation and integration modules within the sprint schedule	Continuous code review, walkthroughs and unit testing; formal technical inspection of high-risk modules before promotion to test	Code review sign-off per merge

SDLC Phase	SDP Contribution	SQAP Contribution	Gate / Sign-off
Integration & Testing	Project Manager coordinates SIT/UAT scheduling and resourcing against the verification-phase timeline	SIT, functional, security/penetration and performance testing executed against defined metrics and severity thresholds	UAT sign-off, zero critical defects
Deployment	Data migration and cutover executed per the deployment schedule and resource plan	Pre-deployment go/no-go review checks acceptance criteria: test pass rate, defect backlog, compliance checklist	Go-live approval
Maintenance	Hypercare support and formal project closure per the SDP's final milestone	Defect tracking, quality-metric trend analysis and post-implementation review at 90 days	Project closure & lessons learned

Table 1 — SDP and SQAP contributions and gate sign-offs by SDLC phase.

7.3 Process Model: The Dual-Track SDLC

The relationship is best expressed as two parallel tracks moving through the same SDLC calendar: an execution track driven by the SDP, and a verification track driven by the SQAP, connected at every phase by a bidirectional gate. Work flows forward from the execution track into the verification track for review; if the review passes, the SDP schedule advances to the next phase; if it fails, work is returned to the execution track for rework before it is re-submitted for review. This means the SDLC does not progress on elapsed time alone — it progresses on verified quality, phase by phase, which is precisely the discipline that prevents an undetected defect in, for example, the interest-calculation engine from being carried forward into integration testing, deployment, and ultimately a customer's repayment schedule. The flowchart below models this dual-track structure for LoanSwap Central, from the initial business requirements through to maintenance and hypercare.

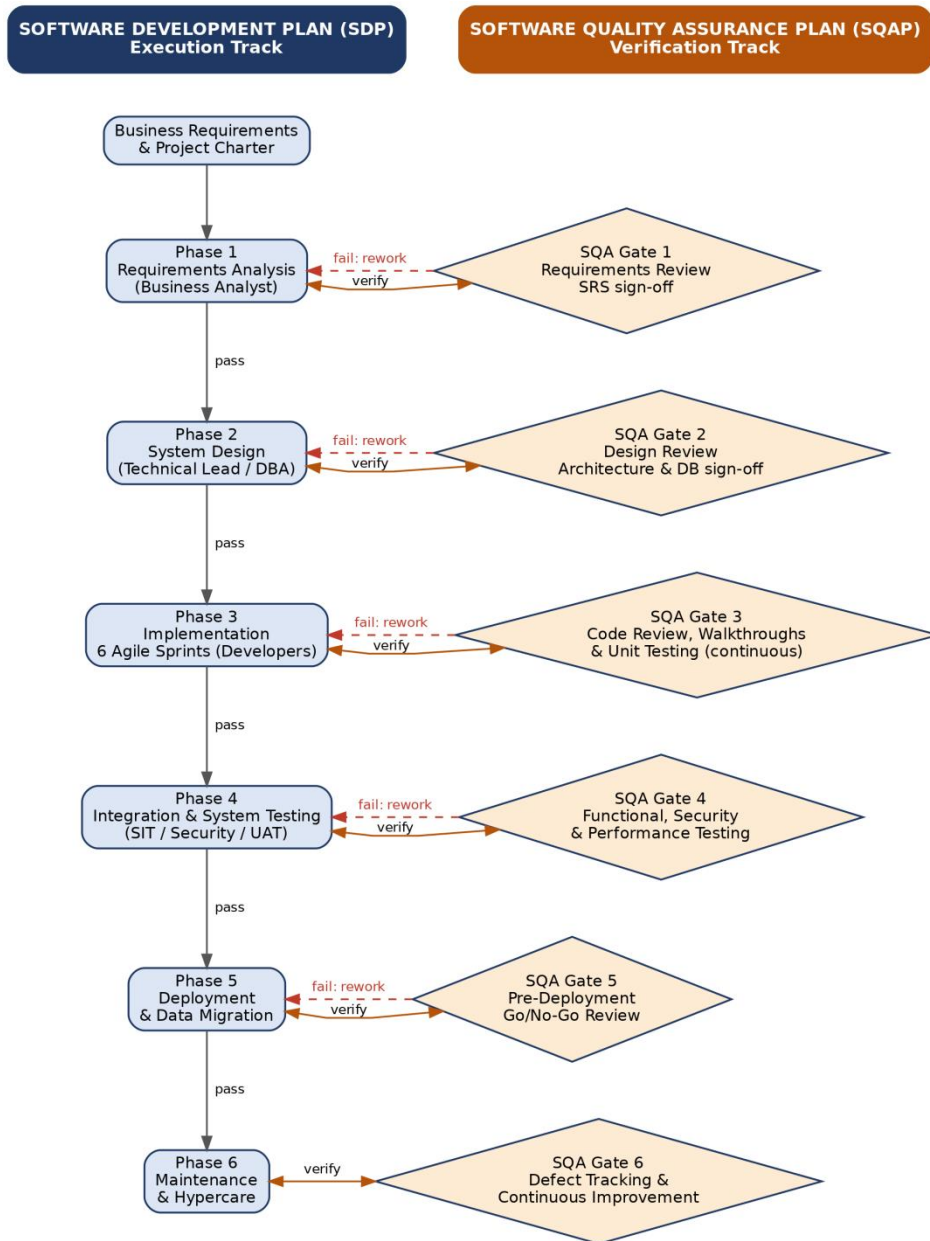


Figure 1 — Dual-track SDLC process model for LoanSwap Central: SDP execution phases (left) gated by SQAP verification activities (right), with rework loops on failure.

7.4 Why the Coupling Matters for LoanSwap Central Specifically

Because LoanSwap Central recalculates repayment obligations and updates two institutions' core ledgers, a defect that escapes one phase does not stay contained within that phase — it propagates. An ambiguous requirement about how a mid-cycle interest-rate change should be applied, if not caught at the Requirements Review gate, becomes an assumption baked into the System Design; if not caught at the Design Review gate, it becomes logic written into the calculation engine during Implementation; if not caught by code review or unit testing, it survives into Integration and System Testing; and if the testing strategy does not specifically target financial calculation accuracy, it survives UAT and reaches production as an incorrect repayment amount — the project's own defect register classifies exactly this scenario as Critical, with a mandatory 24-hour fix and a block on release. The phase-gate coupling between the SDP and SQAP exists to stop this chain at the earliest possible link, which is also the

cheapest point at which to correct it. The same logic applies to the Core Banking API integration risk identified in the risk register: the SDP schedules an early proof-of-concept in Months 2-3, and the SQAP's integration testing activities are what confirm that proof-of-concept actually behaves as required before the six implementation sprints commit to building against it.

Across LoanSwap Central's SDLC, the SDP and SQAP function as a single governance mechanism expressed in two documents: the SDP sets the pace and allocates the resources, while the SQAP decides, at each phase gate, whether that pace is earned. Their coupling is what allows Bank's Steering Committee to make a genuinely informed decision at every milestone — not "is the schedule on track" in isolation, but "is the schedule on track with verified quality behind it." For a system that moves real financial obligations between institutions, that distinction is the entire point of running the two plans together rather than either one alone.

8. Effects If the Project Changes from a 12-Month External Client Project to a 2-Month Internal Organizational Project

8.1 SDP Components That Would Be Simplified

- Shorter project schedule.
- Smaller Work Breakdown Structure (WBS).
- Reduced budget planning.
- Simpler communication plan because fewer stakeholders are involved.
- Simpler risk management and resource allocation.
- Lightweight project documentation.

8.2 SQA Activities That Remain Essential

- Requirements review.
- Design review.
- Code reviews.
- Unit, integration and user acceptance testing.
- Defect tracking and fixing.
- Final quality verification before deployment.

8.3 Why Planning and Documentation Are Still Necessary

Even in a small internal project, planning prevents confusion, keeps the team aligned, defines responsibilities, controls scope, tracks progress, and records important decisions. Documentation also supports future maintenance, upgrades, troubleshooting, onboarding of new team members, and ensures the Bank Loans Swap System remains reliable, secure, and compliant with organizational policies.

9. Consequences of Proceeding Without a Software Development Plan

The SDP is the instrument that converts a business problem — slow, error-prone, branch-dependent loan restructuring — into an executable, resourced, sequenced piece of work. Without it, the project does not become simpler; it becomes unmanaged.

9.1 Loss of Schedule Control and Scope Definition

LoanSwap Central's SDP defines a hybrid delivery model: a phase-gated Analysis and Design stage (Months 1-4), six two-week Agile sprints for Implementation (Months 5-8), and a Structured Verification and Deployment stage (Months 8-12), each tied to a Work Breakdown Structure and an in-scope/out-of-scope boundary (for example, the system swaps and restructures existing facilities but explicitly excludes underwriting new loans). Without an SDP, there is no WBS to decompose the eligibility engine, the recalculation engine, the approval-workflow dashboard and the Core Banking/Credit Bureau integrations into estimable units of work, and no scope boundary to stop those units from expanding. In practice this is how scope creep enters a banking project: a treasury analyst asks, mid-build, for the swap engine to also support cross-currency facilities, and because there is no documented in-scope/out-of-scope line and no Change Control gate to weigh the request against schedule and budget, the request is simply absorbed into the sprint backlog. The 12-month, six-sprint Implementation window silently expands, but the Steering Committee — which the SDP obliges to review milestone status monthly — has nothing to review it against, because there was never a baseline schedule to compare it to.

9.2 Budget Overrun and Resource Misallocation

The SDP allocates the \$950,000 budget across personnel (60%), infrastructure and API licences (15%), quality assurance and security audits (15%) and a contingency buffer (10%), and pairs that budget with a defined team: a Project Manager, Business Analysts, a Technical Lead with four developers, a DBA, a UI/UX designer, a QA Lead with two testers and a Compliance/Security Specialist. Without this plan, a bank commissioning the project has no basis for knowing whether the team assembled is right-sized for a system that must integrate with legacy Core Banking APIs and Credit Bureau services while meeting OWASP Top 10 and Basel III obligations. A common and entirely foreseeable failure mode is under-resourcing quality assurance relative to development — hiring developers to write the swap recalculation and API integration code but treating testing as an activity developers will "also do," rather than budgeting the QA Lead and two dedicated testers the SDP specifies. The visible symptom is a system that appears to progress quickly in early sprints, because untested code is fast to write, followed by a testing phase that discovers the accumulated defects all at once, with no contingency budget set aside to absorb the rework.

9.3 Organisational Confusion and Communication Breakdown

The SDP's Project Organisation section assigns explicit accountability — the Technical Lead and developers own architecture and API implementation, the DBA owns schema design and data migration, the QA Lead owns SQAP execution and defect verification, the Compliance and Risk Liaison owns Basel III/IFRS 9 and data-privacy review — and its Communication Plan ties each stakeholder group to a defined cadence (monthly Steering Committee meetings, daily scrums, milestone phase-gate reviews, quarterly audit summaries). Without this structure, responsibility for decisions that affect two institutions' balance sheets becomes ambiguous. If a developer discovers that the legacy Core Banking API cannot support real-time balance updates and only supports nightly batch reconciliation, there is no defined channel — and no Compliance and Risk Liaison formally engaged from

Month 1 as the SDP requires — through which that discovery reaches the people who must decide whether nightly reconciliation is acceptable for a system computing interbank obligations. The decision either gets made informally by whichever developer encountered the constraint, or it does not get made at all until the discrepancy surfaces in production.

9.4 Unmanaged Technical and Integration Risk

LoanSwap Central's own risk register identifies Core Banking integration failure via legacy APIs as a medium-likelihood, high-impact risk, mitigated in the SDP by an early Months 2-3 API proof-of-concept. This is a direct illustration of what the SDP is for: it converts a known category of banking-system risk into a scheduled activity with a defined outcome, before the six implementation sprints commit developer time to building against an interface that may not behave as assumed. Without an SDP there is no mechanism that forces this proof-of-concept to happen before coding starts. Development teams under time pressure tend to begin building the eligibility and recalculation engines against an assumed API contract, discovering only when they attempt to wire in the real Core Banking system — potentially in Month 8 or 9 rather than Month 2 — that the legacy interface does not expose the fields needed for zero-variance ledger reconciliation. At that point the cost of correcting the assumption is no longer a design conversation; it is a rewrite of completed modules under schedule pressure, which is precisely the dynamic that produces rushed, poorly tested integration code.

9.5 Contractual and Governance Exposure

Bank's own SDP explicitly cites the risk of "CFV-style" project failures — referencing a case in which unreviewed assumptions about requirements, estimates and technical feasibility produced an 18-month delay and significant cost overruns — as the justification for its two-stage Contract Review discipline (a Proposal Draft Review before submission, and a Contract Draft Review after negotiation, to catch unbudgeted scope additions). Skipping the SDP removes this safeguard entirely. A project the size and regulatory sensitivity of LoanSwap Central would then be executed on the basis of informal, unverified assumptions about what "loan swap management" means to Bank's treasury and compliance functions, with no checkpoint to confirm that estimates were built on historical data or that technical feasibility had actually been demonstrated. The project would be exposed to exactly the failure pattern its own planning documentation was written to prevent.

10. Consequences of Proceeding Without a Software Quality Assurance Plan

If the SDP answers what will be built and when, the SQAP answers how Bank will know it was built correctly. LoanSwap Central's SQAP commits the project to ISO/IEC 25010 quality characteristics, ISO/IEC/IEEE 29119-structured testing, IEEE 730 process discipline, layered review and inspection activities, defined quality metrics, a defect severity classification, quality audits and formal acceptance criteria. Removing this plan does not make the underlying software any less complex or any less consequential to two banks' balance sheets — it simply removes the organisation's ability to detect problems before customers, auditors or regulators do.

10.1 Defects Detected Late, at Much Higher Cost

A foundational principle of software engineering — and the stated rationale in Bank's own SQAP for conducting reviews and inspections throughout the SDLC — is that a defect is dramatically cheaper to fix the earlier it is found: a requirements ambiguity caught in a requirements review costs a conversation and a document edit, while the same ambiguity, if it reaches production as an incorrect repayment calculation, costs a hotfix, a defect investigation, a reconciliation exercise against the core ledger, and potentially a customer remediation. LoanSwap Central's SQAP schedules requirements reviews, design reviews, continuous code review during implementation, formal technical inspection of high-risk modules (the interest engine, the core banking link, security) before promotion to test, and a pre-deployment go/no-go review. Without this cadence, the project's only quality signal is whichever ad-hoc testing individual developers choose to perform, and the first systematic check of the software's correctness happens — if at all — immediately before go-live, when there is neither time nor budget contingency to absorb what is found.

10.2 Financial Calculation Errors Reaching Customers

The SQAP's own defect severity table classifies "loan swap produces an incorrect repayment amount" as a Critical defect requiring a fix within 24 hours and blocking release, and one of the project's core quality objectives is zero percent variance in financial reconciliation between the swap engine and the core banking ledger. This is not a hypothetical category — interest and amortisation logic is one of the more error-prone areas of financial software, because it combines day-count conventions, compounding-period edge cases, rate-change effective dates and rounding rules that are easy to get subtly wrong and easy for a developer to believe are correct. Without unit testing of the eligibility and calculation engine, without a code walkthrough of that module before it is merged, and without a formal technical inspection before it is promoted to a test environment — all specific SQAP activities — a rounding error or an incorrect handling of a mid-cycle rate change could pass unnoticed through implementation and surface only when a customer's restructured repayment schedule does not reconcile with what Bank's core ledger records. At that point the error is no longer a code review comment; it is a financial discrepancy across two systems of record, potentially across two institutions, requiring manual reconciliation and disclosure.

10.3 Security Vulnerabilities and Data Exposure

One of LoanSwap Central's stated project objectives is to pass 100% of OWASP Top 10 security checks before go-live, and the SQAP assigns dedicated security testing — authentication, authorisation, data encryption, penetration testing — to a Security Specialist, backed by role-based access control and automated audit trails. A system handling customer financial data and interbank settlement instructions is a realistic target for exactly the

vulnerability classes OWASP catalogues: broken access control that lets a loan officer approve a swap they should be restricted from, or an injection flaw in a reporting interface that exposes customer account data. Without a security testing strategy and an independent penetration test, as the risk register itself anticipates under "cybersecurity breach exposing customer data," these classes of defect are not systematically searched for; they are found only if a bad actor or an accident finds them first. In a banking context, that translates directly into regulatory sanctions, mandatory breach disclosure and reputational damage — the consequence Bank's own risk register rates as low-likelihood but high-impact, which is precisely the profile of risk that structured testing exists to manage down.

10.4 Regulatory and Compliance Failure

Bank's quality objectives commit the project to 100% compliance with Basel III capital adequacy standards, IFRS 9 financial-instrument accounting, and central bank lending directives, verified through a compliance checklist signed off by the Compliance Officer before deployment. Achieving this is not something that can be confirmed by inspecting finished code; it depends on requirements traceability — mapping each regulatory rule to a requirement and each requirement to at least one passed test case, an activity the SQAP makes explicit under its acceptance criteria. Without a documented quality plan there is no requirements traceability matrix, no scheduled compliance review at each phase gate, and no structured way for the Compliance and Risk Liaison to confirm that IFRS 9 impairment treatment or Basel III exposure reporting was actually implemented as specified rather than assumed. The realistic outcome is that non-compliance is discovered by an external or internal audit after deployment, at which point remediation means changing a live system that is already processing real interbank obligations, under regulatory scrutiny, rather than correcting a design document.

10.5 No Objective Basis for Go-Live

The SQAP defines explicit, measurable acceptance criteria for LoanSwap Central: all requirements traced to a passed test case, all critical and high-severity defects resolved and verified, User Acceptance Testing completed by nominated Bank staff with at least a 95% pass rate, security testing confirming no unresolved critical or high vulnerabilities, and performance testing confirming response-time targets under simulated peak load. These criteria are what allow the Steering Committee to make an informed go/no-go decision rather than a hopeful one. Without an SQAP, "ready for production" becomes a subjective judgement made under schedule pressure, with no test pass-rate threshold, no defined defect backlog ceiling and no independent verification that the eligibility, recalculation and approval-routing logic behaves correctly across the range of loan products, currencies and edge cases the system is meant to support. A system can be deployed that looks finished — the user interface is complete and the happy-path scenario works in a demonstration — while the underlying logic has never been systematically tested against the full range of interest-rate and tenor-restructuring combinations Bank's loan book actually contains.

10.6 The Compounding Effect: Two Missing Plans, One Amplified Failure

The SDP and SQAP are not independent safeguards; LoanSwap Central's own documentation makes clear that every SDP milestone phase-gate requires a formal SQA review sign-off before the project is allowed to proceed to the next stage. This means the absence of one plan magnifies the damage caused by the absence of the other. Without an SDP there is no proof-of-concept phase to surface the Core Banking integration constraint early; without an SQAP there is no integration testing discipline to catch the resulting data inconsistency before it reaches production. Without an SDP there is no contingency budget or Change Control process to absorb late-

discovered defects; without an SQAP those defects are discovered even later, when they are most expensive to fix. The table below summarises how specific gaps in each plan translate into concrete failure modes for this project.

Missing Artefact	Typical Failure Mode in LoanSwap Central	Illustrative Consequence
Software Development Plan (SDP)	No documented WBS or phase-gate schedule; Core Banking API integration attempted without a proof-of-concept	Integration discovered to be technically infeasible in Month 9 instead of Month 2-3, forcing a redesign under deadline pressure
Software Development Plan (SDP)	No Contract Review discipline (cf. CFV Corporation case)	Unverified assumptions about scope and effort carried into the build, mirroring the 18-month delay and cost overrun seen at CFV Corporation
Software Quality Assurance Plan (SQAP)	No formal technical inspection of the interest-calculation engine before promotion to test	A rounding or compounding-period defect in the swap recalculation logic reaches UAT or production, producing an incorrect repayment amount — a Critical-severity defect under the project's own classification
Software Quality Assurance Plan (SQAP)	No security testing strategy or OWASP Top 10 checklist	An authorisation flaw in the approval workflow allows a loan officer to approve their own swap request, or customer financial data is exposed
Software Quality Assurance Plan (SQAP)	No requirements traceability or defined acceptance criteria	The Bank's Compliance Officer has no objective basis to confirm Basel III / IFRS 9 reporting rules were implemented, delaying or blocking sign-off at go-live

Table 1 — Illustrative failure modes arising from the absence of an SDP and an SQAP on LoanSwap Central.

11. Conclusion

For a system like LoanSwap Central — one that recalculates real repayment obligations, moves them between institutions, and feeds regulatory disclosures under Basel III and IFRS 9 — the SDP and SQAP are not documentation exercises performed to satisfy a coursework rubric; they are the control mechanisms that stand between a \$950,000, 12-month project and the specific failure modes Bank's own risk register, defect classification and quality objectives describe: late-discovered integration failures, financial reconciliation errors, security vulnerabilities in a system handling customer financial data, and regulatory non-compliance discovered after go-live rather than before it. Beginning development without these plans does not make the project faster; it defers the cost of planning and quality control to a later point in the SDLC where the same problems are harder to see, more expensive to fix, and far more likely to be discovered by a customer, an auditor or a regulator instead of a reviewer. The SDP and SQAP exist precisely because, in a domain where a defect can mean an incorrect repayment amount on a real customer's loan, discovering that defect early is not optional.